MAIN CODE

```
module ALU (
    input  [7:0] A,          // 8-bit Input Operand A
    input  [7:0] B,          // 8-bit Input Operand B
    input  [3:0] ALU_Sel,    // 4-bit Operation Select (Opcode)
    output reg [7:0] ALU_Out  // 8-bit ALU Output Result
);

    always @(*) begin
        case (ALU_Sel)
            // Arithmetic Operations
            4'b0000: ALU_Out = A + B;              // Addition
            4'b0001: ALU_Out = A - B;              // Subtraction
            // 4'b0010: ALU_Out = A * B;           // Multiplication (requires 16-bit output)
            4'b0011: ALU_Out = A / B;              // Division

            // Shift Operations
            4'b0100: ALU_Out = A << 1;             // Logical Shift Left
            4'b0101: ALU_Out = A >> 1;             // Logical Shift Right

            // Rotate Operations (using bit concatenation)
            4'b0110: ALU_Out = {A[6:0], A[7]};     // Rotate Left
            4'b0111: ALU_Out = {A[0], A[7:1]};     // Rotate Right

            // Bitwise Logical Operations
            4'b1000: ALU_Out = A & B;              // Bitwise AND
            4'b1001: ALU_Out = A | B;              // Bitwise OR
            4'b1010: ALU_Out = A ^ B;              // Bitwise XOR
            4'b1011: ALU_Out = ~(A & B);           // Bitwise NAND
```

```
        4'b1100: ALU_Out = ~(A | B);              // Bitwise NOR
        4'b1101: ALU_Out = ~(A ^ B);              // Bitwise XNOR

        // Comparison Operations
        4'b1110: ALU_Out = (A > B) ? 8'd1 : 8'd0;  // Greater Than
        4'b1111: ALU_Out = (A == B) ? 8'd1 : 8'd0; // Equal To

        default: ALU_Out = 8'hXX;                 // Default / Unknown state
      endcase
   end

endmodule
```

TEST BENCH

```
`timescale 1ns/1ps

module tb_ALU;

   reg  [7:0] A, B;
   reg  [3:0] ALU_Sel;
   wire [7:0] ALU_Out;

   reg  [7:0] expected;
   reg  [3:0] opcode;
   integer errors;
   integer passed;
   integer i;

   // Design Under Test
```

```
ALU uut (
    .A(A),
    .B(B),
    .ALU_Sel(ALU_Sel),
    .ALU_Out(ALU_Out)
);

// Self-checking task
task check_alu;
    input [3:0] op;
    input [7:0] a_in;
    input [7:0] b_in;

    reg [7:0] exp_result;

    begin
        A = a_in;
        B = b_in;
        ALU_Sel = op;

        // Independent reference model
        case (op)
            4'b0000: exp_result = a_in + b_in;
            4'b0001: exp_result = a_in - b_in;

            // Opcode 0010 is not specified yet

            4'b0011: begin
                if (b_in != 0)
```

```
            exp_result = a_in / b_in;
        else
            exp_result = 8'hXX;
    end

    4'b0100: exp_result = a_in << 1;
    4'b0101: exp_result = a_in >> 1;

    4'b0110: exp_result =
        {a_in[6:0], a_in[7]};

    4'b0111: exp_result =
        {a_in[0], a_in[7:1]};

    4'b1000: exp_result = a_in & b_in;
    4'b1001: exp_result = a_in | b_in;
    4'b1010: exp_result = a_in ^ b_in;
    4'b1011: exp_result = ~(a_in & b_in);
    4'b1100: exp_result = ~(a_in | b_in);
    4'b1101: exp_result = ~(a_in ^ b_in);

    4'b1110: exp_result =
        (a_in > b_in) ? 8'd1 : 8'd0;

    4'b1111: exp_result =
        (a_in == b_in) ? 8'd1 : 8'd0;

    default: exp_result = 8'hXX;
endcase
```

```
    expected = exp_result;
    #1;

    // Skip unsupported or invalid reference cases
    if ((op == 4'b0010) ||
        ((op == 4'b0011) && (b_in == 0))) begin
        $display(
            "SKIP: opcode=%b A=%h B=%h",
            op, a_in, b_in
        );
    end
    else if (ALU_Out !== expected) begin
        errors = errors + 1;

        $display(
            "FAIL: op=%b A=%h B=%h expected=%h got=%h",
            op, a_in, b_in, expected, ALU_Out
        );
    end
    else begin
        passed = passed + 1;

        $display(
            "PASS: op=%b A=%h B=%h result=%h",
            op, a_in, b_in, ALU_Out
        );
    end
end
```

```
endtask

initial begin
$dumpfile("alu_wave.vcd");
$dumpvars(0, tb_ALU);

    errors = 0;
    passed = 0;
    A = 0;
    B = 0;
    ALU_Sel = 0;

    $display("===== DIRECTED TESTS =====");

    // Addition and subtraction
    check_alu(4'b0000, 8'd0,   8'd0);
    check_alu(4'b0000, 8'd240, 8'd15);
    check_alu(4'b0000, 8'd255, 8'd255);
    check_alu(4'b0001, 8'd10,  8'd3);
    check_alu(4'b0001, 8'd0,   8'd1);

    // Division
    check_alu(4'b0011, 8'd20, 8'd4);
    check_alu(4'b0011, 8'd21, 8'd4);
    check_alu(4'b0011, 8'd10, 8'd0);

    // Shifts and rotations
    check_alu(4'b0100, 8'b10000001, 8'd0);
    check_alu(4'b0101, 8'b10000001, 8'd0);
```

```
check_alu(4'b0110, 8'b10000001, 8'd0);
check_alu(4'b0111, 8'b10000001, 8'd0);

// Logic operations
check_alu(4'b1000, 8'hA5, 8'h3C);
check_alu(4'b1001, 8'hA5, 8'h3C);
check_alu(4'b1010, 8'hA5, 8'h3C);
check_alu(4'b1011, 8'hA5, 8'h3C);
check_alu(4'b1100, 8'hA5, 8'h3C);
check_alu(4'b1101, 8'hA5, 8'h3C);

// Comparisons
check_alu(4'b1110, 8'd9, 8'd4);
check_alu(4'b1110, 8'd4, 8'd9);
check_alu(4'b1111, 8'd7, 8'd7);
check_alu(4'b1111, 8'd7, 8'd8);

$display("===== RANDOMIZED TESTS =====");

// Randomized testing of defined operations
for (i = 0; i < 200; i = i + 1) begin
    A = $random;
    B = $random;
    opcode = $random;

    // Avoid undefined opcode 0010
    if (opcode == 4'b0010)
        opcode = 4'b1000;
```

```
            // Avoid division by zero
            if ((opcode == 4'b0011) && (B == 0))
                B = 8'd1;

            check_alu(opcode, A, B);
        end

        $display("================================");
        $display("PASSED TESTS : %0d", passed);
        $display("FAILED TESTS : %0d", errors);
        $display("TOTAL CHECKS : %0d", passed + errors);

        if (errors == 0)
            $display("RESULT: ALL TESTS PASSED");
        else
            $display("RESULT: TESTS FAILED");

        $display("================================");

        if (errors != 0)
            $fatal(1, "ALU verification failed");

        $finish;
    end

endmodule
```